

BUKU PANDUAN & LAPORAN

TEKNIS SKRIPSI

Judul Proyek: AgriBot - Chatbot Konsultasi Hama & Penyakit Tanaman Hybrid
Tahun: 2026

BAB 1: ARSITEKTUR SISTEM (HYBRID CHATBOT)

1.1 Konsep Pemisahan *Frontend* dan *Backend* (*Decoupled Architecture*)

Sistem AgriBot dibangun menggunakan arsitektur yang memisahkan antarmuka pengguna (*Frontend*) dan logika pemrosesan data (*Backend*). Pemisahan ini bertujuan untuk meningkatkan skalabilitas dan kemudahan pemeliharaan (*maintenance*) sistem.

- **Frontend (PHP, HTML, Vanilla JS):** Bertugas menangani antarmuka pengguna, menampilkan tata letak (*layout*) ruang obrolan, dan menerima input teks dari pengguna. Teknologi PHP digunakan untuk merender struktur halaman web secara dinamis, sementara HTML dan Vanilla JavaScript mengatur interaksi pengguna pada sisi klien (*client-side*).
- **Backend API (Python, Flask):** Berfungsi sebagai otak atau mesin pemroses utama dari chatbot. Flask, sebuah *micro-framework* Python, digunakan untuk membangun *Application Programming Interface* (API) yang menangani pemrosesan bahasa alami (NLP), pencarian kecocokan teks, hingga integrasi dengan layanan *Large Language Model* (LLM) eksternal.

1.2 Komunikasi *Asynchronous* menggunakan JavaScript (*Fetch API / AJAX*)

Agar pengalaman interaksi dengan chatbot terasa mulus tanpa perlu memuat ulang (*refresh*) halaman web setiap kali pesan dikirim, sistem menggunakan metode komunikasi *asynchronous*.

JavaScript (khususnya antarmuka **Fetch API**) digunakan untuk mengirimkan teks pertanyaan pengguna dari *Frontend* menuju *Endpoint* API Flask di *Backend*. Selama *Backend* memproses jawaban, pengguna tetap dapat berinteraksi dengan halaman web. Setelah *Backend* selesai merumuskan jawaban, hasil tersebut dikembalikan ke *Frontend* dalam format JSON dan secara dinamis disisipkan ke dalam antarmuka obrolan menggunakan manipulasi *Document Object Model* (DOM).

1.3 Alur Kerja Sistem (*Workflow*)

Berikut adalah urutan alur kerja dari saat pengguna mengetik pesan hingga *bot* merespons:

1. **Input Pengguna:** Pengguna mengetikkan pertanyaan seputar penyakit tanaman pada antarmuka web dan menekan tombol kirim.
2. **Transmisi Data:** JavaScript menangkap teks tersebut dan mengirimkannya ke API Flask (misalnya ke *endpoint /api/chat*) melalui HTTP POST *request*.
3. **Pemrosesan Backend (NLP):** Flask menerima teks tersebut, melakukan pra-pemrosesan teks, dan membandingkannya dengan basis pengetahuan (*knowledge base*) lokal menggunakan algoritma kecerdasan buatan.
4. **Evaluasi Hybrid:** Jika pertanyaan dikenali dengan tingkat keyakinan tinggi, *bot* akan langsung menjawab dari *database* lokal. Jika tidak dikenali, pertanyaan diteruskan ke model AI generatif (*fallback*).
5. **Pengembalian Respons:** Jawaban yang telah dirumuskan (baik dari lokal maupun AI generatif) dikirim kembali ke *Frontend* dalam format JSON.
6. **Tampilan Output:** JavaScript memproses JSON tersebut dan menampilkan teks balasan pada layar obrolan pengguna.

BAB 2: LANDASAN TEORI AI & NLP

2.1 Tahap Pra-pemrosesan Teks (*Text Preprocessing*)

Sebelum teks pertanyaan pengguna dianalisis, teks tersebut harus dibersihkan dan distandardisasi melalui beberapa tahapan NLP:

- **Pembersihan Karakter (Regex):** Menggunakan *Regular Expression* untuk menghapus tanda baca, simbol, angka, dan karakter khusus yang tidak relevan, sehingga menyisakan huruf alfabet saja.
- **Stopword Removal (Library Sastrawi):** Menghilangkan kata-kata hubung atau kata umum yang tidak memiliki makna krusial dalam pencarian konteks penyakit tanaman (contoh: "yang", "dan", "di", "ke", "adalah").
- **Stemming (Library Sastrawi):** Mengembalikan kata-kata berimbuhan ke bentuk kata dasarnya (contoh: "menguning" menjadi "kuning", "pembusukan" menjadi "busuk"). Hal ini memastikan algoritma tidak membedakan kata yang memiliki akar makna yang sama.

2.2 Representasi Teks dan Algoritma Pencarian Kecocokan

Sistem menggunakan pendekatan *Machine Learning* klasik untuk mencari kecocokan antara pertanyaan pengguna dengan *database* penyakit tanaman secara konseptual:

- **TF-IDF (*Term Frequency - Inverse Document Frequency*):** Algoritma ini bertugas mengubah kata-kata (teks) menjadi vektor angka (*vektorisasi*). Konsep utamanya adalah memberikan bobot tinggi pada kata-kata yang sering muncul dalam suatu kalimat/dokumen tertentu, tetapi jarang muncul di dokumen lain. Misalnya, kata "bercak"

dan "coklat" akan mendapat bobot lebih tinggi daripada kata "daun", karena "daun" mungkin muncul di hampir semua keluhan penyakit.

- **Cosine Similarity:** Setelah teks diubah menjadi vektor angka oleh TF-IDF, *Cosine Similarity* bertugas membandingkan vektor pertanyaan pengguna dengan vektor-vektor yang ada di *database*. Secara konseptual, algoritma ini tidak mengukur jarak fisik antar vektor, melainkan **mengukur sudut kemiripan arahnya**. Semakin searah vektor pertanyaan dengan vektor *database*, semakin tinggi skor kecocokannya (mendekati nilai 1).

2.3 Konsep *Hybrid Fallback*

Kelemahan chatbot berbasis *knowledge base* lokal adalah ketidakmampuannya menjawab pertanyaan yang sama sekali baru atau di luar *database*. Untuk mengatasi ini, AgriBot menerapkan konsep **Hybrid Fallback**.

Sistem menetapkan sebuah ambang batas (*threshold*) kemiripan sebesar **0.20**.

- Jika skor *Cosine Similarity* dari pertanyaan pengguna berada di atas **0.20**, bot akan menampilkan jawaban baku dari *database* lokal (MySQL).
- Jika skor berada di bawah **0.20** (menandakan bot lokal tidak paham atau tidak menemukan kecocokan yang relevan), sistem secara otomatis "melempar" (*fallback*) pertanyaan tersebut (*prompt*) ke API eksternal, yaitu **Gemini 2.5 Flash API**. Gemini kemudian akan menghasilkan jawaban natural dan dinamis berdasarkan pengetahuan luasnya mengenai agrikultur, memastikan pengguna tidak pernah menerima pesan *error* atau jalan buntu.

BAB 3: METODOLOGI PENGUJIAN

3.1 Pengujian Fungsional (*Black Box Testing*)

Pengujian ini berfokus pada evaluasi fungsionalitas sistem dari sudut pandang pengguna tanpa perlu melihat struktur kode internal.

- **Fokus Pengujian:** Menguji elemen *User Interface* (UI) dan *User Experience* (UX) pada web antarmuka.
- **Skenario:** Memastikan tombol "Kirim" berfungsi, pesan tampil pada posisi yang benar, *loader/loading animation* muncul saat menunggu balasan dari Flask, dan sistem mampu menangani input teks kosong atau input berupa karakter aneh tanpa menyebabkan sistem *crash* (validasi input).

3.2 Pengujian Performa Model (*Confusion Matrix*)

Untuk mengukur seberapa pintar sistem lokal dalam mengklasifikasikan penyakit tanaman, digunakan pengujian matriks kebingungan (*Confusion Matrix*). Pengujian ini tidak hanya mengandalkan metrik **Akurasi** (*Accuracy*), melainkan berfokus pada **Precision, Recall, dan F1-Score**.

- **Alasan Penggunaan Metrik Khusus:** Dalam kasus nyata, *dataset* keluhan hama dan penyakit tanaman seringkali **tidak seimbang (*imbalanced dataset*)**. Misalnya, kasus "Kutu Putih" mungkin sangat banyak (80% data), sedangkan kasus "Virus Gemini" sangat sedikit (20% data).
- Jika menggunakan metrik Akurasi pada data yang *imbalanced*, model yang menebak "Kutu Putih" terus-menerus akan tetap terlihat memiliki akurasi tinggi (80%), namun gagal total mengenali "Virus Gemini".
- Oleh karena itu, **Precision** digunakan untuk mengukur seberapa akurat tebakan positif dari *bot*, **Recall** mengukur seberapa banyak kasus penyakit aktual yang berhasil ditebak oleh *bot*, dan **F1-Score** memberikan nilai rata-rata harmonis antara keduanya, sehingga memberikan evaluasi yang jauh lebih adil dan relevan untuk *dataset* penyakit tanaman.

BAB 4: PANDUAN INSTALASI & PENGGUNAAN (USER MANUAL)

Bagian ini memuat panduan langkah demi langkah untuk menjalankan aplikasi AgriBot di lingkungan lokal (*localhost*).

4.1 Persiapan Database MySQL via XAMPP

1. Unduh dan instal aplikasi **XAMPP**.
2. Buka aplikasi **XAMPP Control Panel**.
3. Klik tombol **Start** pada modul **Apache** (untuk menjalankan server web jika diperlukan untuk PHP) dan modul **MySQL** (untuk menyalakan *database*).
4. Buka *browser* (Chrome/Firefox) dan ketikkan alamat: `http://localhost/phpmyadmin`.
5. Buat *database* baru (misalnya dengan nama `agribot_db`) dan *import* file `.sql` yang telah disediakan pada *source code*.

4.2 Instalasi *Dependency* Python

Pastikan Python telah terinstal di komputer Anda. Buka terminal atau *Command Prompt* (CMD) di dalam *folder project backend*, lalu jalankan perintah berikut untuk menginstal pustaka yang dibutuhkan:

```
pip install flask flask-cors scikit-learn sastrawi google-genai pandas mysql-connector-python
```

4.3 Menjalankan Server Backend (API)

Setelah *database* menyala dan *dependency* terinstal, Anda harus menyalakan otak dari *chatbot* ini:

1. Buka Terminal / CMD dan arahkan ke dalam direktori *folder backend* (tempat file `app.py` berada).

2. Jalankan perintah berikut:
`python app.py`
3. Terminal akan menampilkan teks informasi bahwa server Flask telah berjalan, biasanya di alamat `http://127.0.0.1:5000`. **Jangan tutup terminal ini** selama Anda masih ingin menggunakan *chatbot*.

4.4 Mengakses Web Konsultasi dan Web Admin

1. Letakkan *folder frontend* proyek ini ke dalam folder `htdocs` yang ada di direktori instalasi XAMPP Anda (misal: `C:\xampp\htdocs\agribot-web`).
2. Buka *browser* pilihan Anda.
3. **Halaman Konsultasi (User):** Ketik URL `http://localhost/agribot-web/index.php` untuk mulai melakukan *chatting* dengan AgriBot.
4. **Halaman Admin:** Ketik URL `http://localhost/agribot-web/admin.php` untuk mengelola *database* penyakit dan respons *chatbot* (tambah, edit, hapus data pengetahuan).